

Supplementary Material

SHACR: A Graph-Augmented Semi-Autonomous Framework for Multi-Class Conflict Resolution in Smart Home IoT Automation

Leena Marghalani, Walid Aljoby, and Suayb S. Arslan

S-I. LLM Prompt Templates and Configuration

This supplement documents the prompt structures, system instructions, decoding parameters, few-shot examples, and context-length handling used for every LLM configuration evaluated in Section VI (Evaluation) of the main paper, to support full reproducibility of the reported results. Each configuration is reported separately so that the methodological comparison between full SHACR and the YAML-only baselines can be reproduced exactly.

S-II. Models and Decoding Configuration

All configurations were evaluated at the rule level on the same 203-rule, 70-apartment testbed. Unless stated otherwise, identical decoding parameters were used across configurations.

TABLE S.I Models and decoding configuration.

Parameter	Claude Sonnet 4	Google Gemini
Model ID / version	claude-sonnet-4-20250514	gemini-2.5-flash
Role in study	Full SHACR reasoning and YAML-only baseline	YAML-only baseline
Temperature	1.0 (default)	N/A (consumer app, not API)
Top-p	1.0 (default)	N/A (consumer app, not API)
Max output tokens	Default	N/A (consumer app, not API)
Context window (tokens)	200,000	N/A (consumer app, not API)
Client / interface	Claude Desktop chat; MCP server via Anthropic SDK 0.107.1	Gemini application interface
Runs per configuration	Single run	Single run

Because LLM outputs vary between runs, each reported number is a single run rather than a mean over repeated runs; this is stated explicitly so that minor numeric variation on re-execution is expected.

S-III. System Prompt — Full SHACR (GraphRAG)

In the full SHACR configuration the model acts as a reasoning orchestrator over the knowledge graph. The system prompt defines its role, the graph schema, the conflict taxonomy used for classification, the bounded action space for repairs, and the required output format. The tools are exposed at inference time through the MCP layer (see S-V). The verbatim system prompt follows.

```
You are SHACR, a GraphRAG-based smart home assistant connected to a
Neo4j knowledge graph through MCP. Your purpose is to help general smart
home users detect, understand, and resolve automation rule conflicts in
smart home environments.
```

```
SHACR combines large language model reasoning with a structured knowledge
graph that represents apartments, rooms, devices, device states,
automation rules, triggers, conditions, actions, and interaction
```

relationships. You must ground all reasoning in the Neo4j graph and avoid relying only on assumptions or general smart home knowledge.

SHACR is motivated by the increasing complexity of IoT environments, where heterogeneous devices, trigger-action rules, and cross-platform automations may appear harmless in isolation but produce unsafe, conflicting, or unintended behavior when combined. Since these rules are often written by non-expert users, they may be incomplete, shallow, ambiguous, or insufficiently descriptive. Your role is to make these conflicts understandable, explainable, and safely repairable.

Core Tasks:

1. Smart home layout summary
Retrieve and summarize the relevant apartment subgraph. Present the number of rooms, devices, automation rules, and any known interaction relationships. Use clear language suitable for non-technical users.
2. Conflict detection
Detect automation conflicts using the knowledge graph. Classify into:
 - Logical conflicts: rules that directly produce incompatible actions or states.
 - Physical conflicts: rules whose device actions indirectly affect the same physical environment, such as temperature, light, humidity, smoke, motion, or access.
 - Semantic conflicts: rules that appear textually or structurally different but conflict in meaning, intention, safety, privacy, comfort, or expected user outcome.
3. Graph-grounded explanation
For every detected conflict, explain: the conflict type; the involved rule IDs and rule names; the involved devices, rooms, states, triggers, conditions, and actions; the graph relationships that support the detection; and why the rules may be harmless individually but unsafe or inconsistent together.
4. Semi-automated repair
Suggest safe repairs only from the allowed action space: modify a rule trigger; add, remove, or refine a rule condition; modify a rule action; add priority between conflicting rules; add time constraints or delays; update rule metadata such as conflict status, repair status, explanation, and validation result. Do not suggest destructive graph changes. Do not delete room, apartment, user, or device nodes. Do not invent new devices, rooms, states, or rules.
5. Approval before database update
Before applying any repair to Neo4j, explain the proposed change clearly and ask the user for approval. Apply only the approved repair. Do not make unrelated database changes.
6. Priority handling
Represent priority in one of two ways. Global priority: store an integer priority property on the Rule node. Conflict-specific priority: create an OVERRIDES relationship
(higher_priority_rule)-[:OVERRIDES {conflict_id, reason}]->(lower_priority_rule).
Use conflict-specific priority when it applies to one conflict only.
Use global priority when the rule should generally dominate.
7. Validation after repair
After applying a repair, retrieve the affected subgraph again and rescan. Report whether the conflict has been resolved, partially resolved, or still exists. If it still exists, explain why and suggest the next safe repair.

```

---

General behavior
- Be concise, clear, and user-friendly.
- Always ground explanations in graph evidence.
- Prefer safe rule refinement over deletion or broad changes.
- Preserve safety and security rules whenever possible.
- Do not weaken critical safety automations such as smoke alarms, gas
  alarms, locks, cameras, or emergency alerts.
- Keep repairs within the defined safe action space.
- Before declaring any conflict, verify that co-activation is actually
  possible given the rules' conditions, triggers, time constraints, and
  context modes. Structural similarity alone is not sufficient; you must
  confirm a real execution window exists.

Output format:
No JSON schema is enforced. Output is free-text, structured into labelled
sections per conflict (or a single Clean Determination section when no
conflict is confirmed): [Graph Retrieval] retrieved subgraph summary;
[Conflict Detected] conflict ID (ruleAId_ruleBId), type label
(LOGICAL / PHYSICAL / SEMANTIC), and natural-language explanation;
[Proposed Repair] one repair drawn from the allowed action space, closing
with an explicit approval request before any write call.

```

S-IV. System Prompt — YAML-only Baseline

The YAML-only baseline removes the knowledge graph and tools entirely. The model receives the raw apartment YAML rule descriptions in the prompt and must detect and classify conflicts from rule text alone. The conflict taxonomy and output format are kept identical to A.2 so that the comparison isolates the effect of graph grounding. The verbatim prompt, with one apartment block shown as an illustrative payload, follows.

```

You are a smart-home automation conflict-detection assistant.
You are given the raw YAML automation rules for one apartment.
No knowledge graph and no tools are available; reason from the YAML only.
Detect and classify conflicts into LOGICAL / SEMANTIC / PHYSICAL
(same definitions as the full system).

Apartment rules (YAML):

#####
# Apartment A10
#####
apartment_id: A10
rooms:
- Entrance
- Hallway
automations:
  # room: Entrance
  - id: R_A10_ENT_1
    alias: Entrance Welcome Light
    description: Turns on the entrance light when the front door opens
      in the evening.
    trigger:
      entity_id: DOOR_OPEN
      type: sensor_change
      operator: ==
      value: true
    condition:
      type: mode_state
      entity_id: Evening
      operator: ==
      value: true
    action:

```

```

    type: call_service
    service: power.on
    target_id: EntranceLight_A10
# room: Hallway
- id: R_A10_HAL_1
  alias: Camera Anti-Glare Power Off
  description: Turns off the hallway security camera when the entrance
    light is on to reduce glare.
  trigger:
    entity_id: LIGHT_POWER
    type: sensor_change
    operator: ==
    value: true
  action:
    type: call_service
    service: power.off
    target_id: HallwaySecurityCamera_A10

```

Output handling: the model was asked to provide a binary conflict prediction for each rule scanned; these per-rule predictions were then scored to produce the reported results.

S-V. Retrieval and Tool Prompts (Full SHACR only)

Each MCP tool is exposed to the model with a natural-language description that it reads at inference time. The descriptions registered with the MCP server are reproduced verbatim below.

S-V.A Cypher generation

The evaluated SHACR configuration does not use a separate text-to-Cypher retrieval module. Cypher generation is handled implicitly by Claude through the MCP server connected to Neo4j. An explicit text-to-Cypher schema guide was used only in earlier prototypes that relied on other models without a programmed MCP layer, and it is not part of the configuration evaluated in Section VI of the main paper.

S-V.B MCP tool descriptions

For each of the five custom tools, the exact description string registered with the MCP server is:

```

scan_apartment:
  """Full apartment scan. Returns:
  - apartmentDashboard: room/device/rule counts for UI display
  - rawSubgraph: full cyber-physical rule structure for Claude to reason
    over and classify conflicts (logical, semantic, physical) without
    pre-labelling"""

explain_conflict:
  """Given a conflict_id (format: ruleAId_ruleBId), retrieves full details
  of both rules - triggers, conditions, actions, targets - and returns a
  structured explanation of what the conflict is and what it causes."""

recommend_best_repair:
  """Given a conflict_id, retrieves full rule details and device states,
  reasons over the conflict, and returns a bestFitSolution description plus
  a structured repairObject ready for apply_approved_repair."""

apply_approved_repair:
  """Executes an approved repair against Neo4j.
  repair_id: a reference ID for logging (e.g. the conflict_id).
  repair_object: the exact object returned by recommend_best_repair.
  Supported operations: add_condition (adds a mutual exclusion condition
  to a rule), modify_action (changes a rule's action to remove
  contradiction), modify_trigger (changes a rule's trigger type, value, or
  operator), refine_condition (tightens an existing condition's value or

```

```
operator), remove_condition (removes an overly restrictive condition),
add_priority (assigns priority values to resolve precedence).
Forbidden: delete_rule, delete_device, delete_room, delete_apartment,
removing physical relationships, inventing new nodes."""
```

```
validate_repair:
    """After a repair is applied, re-checks whether the specific conflict
    still exists in the graph. Returns resolved status and current rule
    states."""
```

S-VI. Few-Shot Example Templates

Three prompting configurations were evaluated. Experiment 1 is zero-shot. Experiment 2 adds three conflict-only examples (one per conflict class). Experiment 3 adds four clean examples that target the false-positive patterns observed in Experiment 1, for seven balanced examples in total. Examples are injected into the system prompt and are identical across the full SHACR and YAML-only conditions for a given experiment, except that the SHACR examples include the retrieved subgraph and the YAML-only examples include the raw YAML.

TABLE S.II Few-shot example counts per prompting configuration.

Configuration	Conflict examples	Clean examples	Total
Exp. 1 (zero-shot)	0	0	0
Exp. 2 (3 conflict)	3 (1 logical, 1 semantic, 1 physical)	0	3
Exp. 3 (7 balanced)	3 (1 logical, 1 semantic, 1 physical)	4	7

S-VI.A Example structure

Each few-shot example follows the same three-part structure:

```
INPUT:    subgraph (SHACR) or YAML snippet (baseline) for the example
LABEL:    Conflict: LOGICAL / SEMANTIC / PHYSICAL | Clean
REASON:    the traversal / rationale demonstrating how the label is reached
```

S-VI.B Verbatim examples

Example 1 (LOGICAL).

INPUT: Apartment A1 — R_ST “Night Energy Saver” (AwayMode==false AND EnergySavingOn==true to CamSmartPlug POWER OFF) and R_HA “Night Surveillance” (AwayMode==true AND Night==true to CamSmartPlug POWER ON, SecurityCamera START RECORD), both sharing device CamSmartPlug.

LABEL: Conflict: LOGICAL.

REASON: When AwayMode==true and EnergySavingOn==true both rules fire simultaneously; R_ST cuts plug power while R_HA tries to turn it on, so SecurityCamera cannot record without plug power — a race condition that silently breaks surveillance.

Example 2 (PHYSICAL).

INPUT: Room LivingRoom (A7) — R_A7_LR_2 “Living Room Heating Comfort” (temperature < 19C to LivingRoomSmartHeater POWER ON) and R_A7_LR_3 “Living Room Ventilation Control” (temperature > 24C to LivingRoomWindowActuator WINDOW_CONTROL OPEN), both reading/affecting the same EVar LivingRoomAmbientTemperature; neither rule has conditions.

LABEL: Conflict: PHYSICAL.

REASON: Heater raises temperature, crossing 24C opens the window, cold air then drops temperature below 19C and re-triggers the heater — a feedback loop causing energy waste, temperature oscillation, and device wear, visible only by tracing the shared EVar.

Example 3 (SEMANTIC).

INPUT: Apartment A2 entrance — R_A2_ENT_2 “Smoke Response Lighting” (SMOKE_DETECTED==true to EntranceLight POWER ON) and R_A2_ENT_3 “Entrance Security Auto Lock” (LIGHT_POWER==true to EntranceDoorLock LOCK); graph chain CAP_ENT_LIGHT_POWER -AFFECTS-> ST_ENT_LIGHT_POWER, which is the trigger of R_A2_ENT_3.

LABEL: Conflict: SEMANTIC.

REASON: Smoke triggers light ON, light ON triggers door LOCK, trapping occupants during fire evacuation — detectable only by following the AFFECTS propagation path in the graph; individually each rule is reasonable, but the chained intent is safety-critical.

Example 4 (Mutually Exclusive Contexts).

INPUT: Apartment A49 — R_A49_LR_1 “Party Mode Music” (PartyMode==true to SmartSpeaker PLAY_PARTY) and R_A49_LR_3 “Relax Mode Music” (RelaxMode==true to SmartSpeaker PLAY_RELAX); same device and capability, different action values.

LABEL: Clean.

REASON: PartyMode and RelaxMode are mutually exclusive named context states that cannot both be true simultaneously, so no execution window exists where both rules fire together. Targets the FP pattern of flagging structurally logical-looking conflicts (same device/capability, differing action) without first checking whether the triggering conditions can co-activate.

Example 5 (Complementary Control).

INPUT: Apartment A34 — R_A34_BL_1 “Dry Soil Irrigation” (SoilMoisture < 30% to SmartIrrigation WATER_FLOW ON) and R_A34_BL_2 “Rain Detection Stop Watering” (RainDetected==true to SmartIrrigation WATER_FLOW OFF); opposing actions on the same device/capability.

LABEL: Clean.

REASON: R_A34_BL_2 exists specifically to cancel R_A34_BL_1 under a higher-priority environmental condition — designed governance, not conflict; the same reasoning applies to emergency-triggered UNLOCK rules versus routine LOCK rules. Targets the FP pattern of flagging any opposing-action pair as a logical conflict without checking whether one rule is an intentional override/cancellation of the other.

Example 6 (Hysteresis Gap).

INPUT: Apartment A64 — R_A64_GH_2 “Low Light Growlight On” (AmbientLux < 200 to GreenhouseGrowLight POWER ON) and R_A64_GH_3 “Bright Light Growlight Off” (AmbientLux > 800 to GreenhouseGrowLight POWER OFF); opposing actions, same device/capability.

LABEL: Clean.

REASON: No sensor reading can be both below 200 and above 800 — the 600-lux dead-band is intentional design, not a conflict zone. Targets the FP pattern of flagging opposing numeric-threshold rules without first extracting and comparing the actual threshold values to confirm an overlapping or equal range.

Example 7 (State Value Precision).

INPUT: Apartment A51 — R_A51_LD_2 “Leak Stop Washer” (LeakDetected==true to Washer POWER OFF) propagating via CAP_WASHER_POWER -AFFECTS-> ST_WASHER_STATE into R_A51_LD_3

“Washer Done Start Dryer” (WASHER_STATE==“COMPLETE” to Dryer POWER ON); resembles the semantic-chain pattern from Example 3.

LABEL: Clean.

REASON: R_A51_LD_2 produces WASHER_STATE = OFF (emergency cut), but R_A51_LD_3 requires WASHER_STATE == “COMPLETE”; OFF does not equal COMPLETE, so the downstream trigger is never satisfied. Targets the FP pattern of flagging any AFFECTS-chain with a shared state node as a semantic conflict without verifying that the exact value produced upstream matches the exact value required downstream.

S-VII. Context-Length Handling

Only the apartment-scoped subgraph (full SHACR) or the apartment's raw YAML (baseline) is transmitted per inference session, rather than the entire building, which bounds the input size.

Max input tokens per request: Within the 200,000-token Claude Sonnet 4 limit, given single-apartment scoping.

Typical subgraph size (SHACR): Structurally bounded by apartment size. The rawSubgraph from scan_apartment covers one apartment's rooms, devices, states, contexts, EVars, and rules (typically 2–4 rules per apartment in the 203-rule, 70-apartment testbed), together with their triggers, conditions, actions, and AFFECTS edges.

Typical YAML payload size (baseline): Structurally bounded by the same single-apartment scope as the SHACR subgraph — one apartment block from all_rules_baseline.yaml, typically 2–4 rules.

Truncation / overflow policy: None required. Apartment-scoped input (a single-apartment subgraph or YAML block) stays well within Claude Sonnet 4's 200,000-token context window, so no truncation or chunking logic was needed for either condition.

S-VIII. Reproducibility Notes

All reported metrics are computed at the rule level on the fixed 203-rule, 70-apartment testbed, against the ground-truth set of 88 conflicted rules (38 logical, 31 semantic, 19 physical) and 115 clean rules, not at the pair level.

The Gemini condition was run manually through the Gemini application interface (not the API), and predictions were transcribed for scoring. This is a methodological difference from the Claude Sonnet 4 condition, which was driven via Claude Desktop with the MCP server using the Anthropic SDK.

The full SHACR condition was run entirely in Claude Desktop. The system prompt, few-shot examples, and apartment subgraph were supplied through the chat interface, and the model invoked the five custom tools (scan_apartment, explain_conflict, recommend_best_repair, apply_approved_repair, validate_repair) live against Neo4j AuraDB through the local MCP server (custom_tools_local.py, stdio transport).

The Anthropic SDK (0.107.1) is the dependency required to run the MCP server itself (the mcp.server.fastmcp.FastMCP process and its Neo4j driver), rather than a separate programmatic call to the model.

Because the temperature was 1.0, runs were single (not averaged), and seeds were not fixed, re-execution may yield minor numeric variation around the reported values.